

# Rapport d'analyse statique

## UnCrackable Level 2

Soukaina Bachir

26 avril 2026

### 1. Informations générales

|                     |                              |
|---------------------|------------------------------|
| APK                 | UnCrackable-Level2.apk       |
| Package             | sg.vantagepoint.uncrackable2 |
| Bibliothèque native | libfoo.so                    |
| Secret trouvé       | Thanks for all the fish      |
| Outils              | JADX, Ghidra                 |

### 2. Permissions et composants exportés

**Permissions :** Aucune

**Composants exportés :** MainActivity

### 3. Analyse Java avec JADX

#### MainActivity.java

```
static {
    System.loadLibrary("foo"); // Charge libfoo.so
}

private native void init(); // Anti-debug

public void onCreate(Bundle b) {
    init();
    // Root detection
    if (b.a() || b.b() || b.c()) {
        a("Root detected!");
    }
    // Debug detection
    if (a.a(getApplicationContext())) {
        a("App is debuggable!");
    }
    // Anti-debug loop
    while (!Debug.isDebuggerConnected()) {
        sleep(100);
    }
    this.m = new CodeCheck();
}
```

## CodeCheck.java

```
public class CodeCheck {  
    private native boolean bar(byte[] bArr);  
  
    public boolean a(String str) {  
        return bar(str.getBytes());  
    }  
}
```

## 4. Comment j'ai extrait le secret

Voici les étapes que j'ai suivies pour trouver le secret :

### Étape 1 : Extraire la bibliothèque native

J'ai décompressé l'APK avec la commande :

```
unzip UnCrackable-Level2.apk -d extrait/
```

Puis j'ai localisé `libfoo.so` dans le dossier `lib/x86/`.

### Étape 2 : Ouvrir `libfoo.so` dans Ghidra

J'ai lancé Ghidra, créé un nouveau projet et importé `libfoo.so`. J'ai ensuite lancé l'analyse automatique.

### Étape 3 : Trouver la fonction JNI

Dans Ghidra, j'ai cherché un nom commençant par `Java_` et j'ai trouvé :

```
Java_sg_vantagepoint_uncrackable2_CodeCheck_bar
```

C'est la fonction native qui vérifie le secret.

### Étape 4 : Analyser le pseudo-code

Dans le pseudo-code décompilé, j'ai repéré un appel à `strncmp`. Cette fonction compare l'entrée utilisateur avec une chaîne cachée.

### Étape 5 : Trouver la chaîne hexadécimale

J'ai trouvé une valeur hexadécimale dans le code :

```
6873696620656874206c6c6120726f6620736b6e616854
```

### Étape 6 : Décoder l'hexadécimal

J'ai converti cette valeur en ASCII (avec Python) :

```
hex_data = "6873696620656874206c6c6120726f6620736b6e616854"  
print(bytes.fromhex(hex_data).decode("ascii"))
```

Résultat obtenu : `hsif eht lla rof sknahT`

## Étape 7 : Inverser la chaîne

J'ai inversé la chaîne pour obtenir le secret final :

```
s = "hsif eht lla rof sknahT"  
print(s[::-1])
```

Résultat : Thanks for all the fish

## 5. Flux de l'application

```
Input Utilisateur  
  ↓  
MainActivity.verify()  
  ↓  
CodeCheck.a(String)  
  ↓  
CodeCheck.bar(byte[]) ← NATIVE  
  ↓  
libfoo.so → strcmp() → hex → ASCII → inverser  
  ↓  
Thanks for all the fish
```

## 6. Constats de sécurité

### Constat 1 : Secret dans librairie native (Critique)

**Localisation :** libfoo.so

**Remédiation :** Obfusquer le code natif

### Constat 2 : Anti-debug fork/ptrace (Moyen)

**Localisation :** init() dans libfoo.so

**Remédiation :** Ajouter plusieurs couches d'anti-debug

### Constat 3 : Root detection faible (Moyen)

**Localisation :** MainActivity.onCreate()

**Remédiation :** Utiliser SafetyNet

## 7. Conclusion

Le secret a été extrait avec succès en suivant les étapes :

1. Décompilation avec JADX
2. Extraction de libfoo.so
3. Analyse avec Ghidra Repérage de strcmp
4. Extraction de la valeur hexadécimale
5. Conversion hex → ASCII
6. Inversion de la chaîne

**Thanks for all the fish**

**Lien dépôt :** [https://github.com/bachirsoukaina/LAB5\\_APK-Analysis-UnCrackable-Level2](https://github.com/bachirsoukaina/LAB5_APK-Analysis-UnCrackable-Level2)